

# Day 1: Foundations of Parallel Computing & Concurrency in Python

---

**Duration:** 4 hours (theory + live demos)

**Prerequisites:** Basic Python programming

**Demos folder:** [day1/demo/](#)

---

## Table of Contents

1. [Data Serialization](#)
  2. [Multiprocessing — First Steps](#)
  3. [Multithreading — First Steps](#)
  4. [Processes vs Threads and the Global Interpreter Lock](#)
  5. [Getting Started with Parallel Computing](#)
  6. [Performance Evaluation of Parallel Programs](#)
- 

## 1. Data Serialization

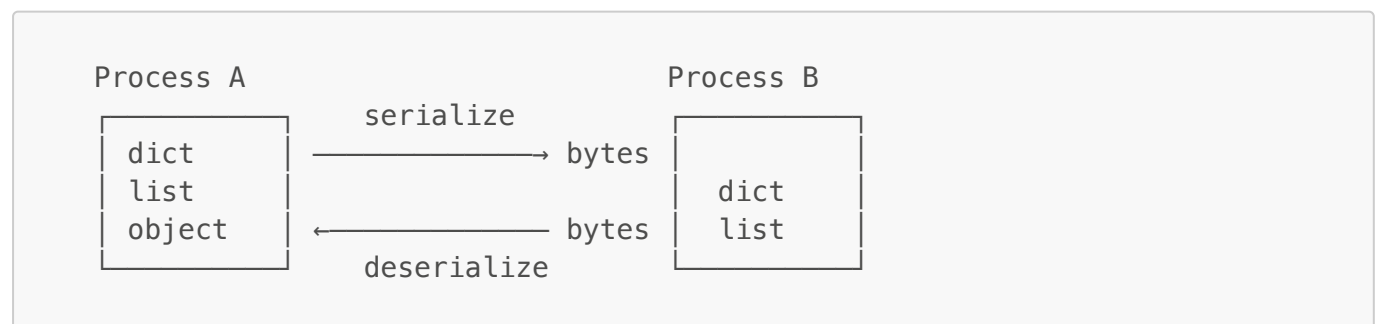
### Why Start Here?

Before we write parallel programs, we need to understand **how data moves** between processes. When you run two separate Python processes, they do **not** share memory. So how does one process send a Python dictionary to another?

The answer is **serialization** — converting an in-memory Python object into a sequence of bytes (or text) that can be:

- Written to a file
- Sent over a network socket
- Passed through a pipe between processes

The receiving end **deserializes** (decodes) those bytes back into a live Python object.



Python gives us two built-in serialization modules: **JSON** and **Pickle**.

---

### 1.1 JSON Serialization

**JSON** (JavaScript Object Notation) is a lightweight, text-based, human-readable format for data exchange. It is the lingua franca of web APIs and configuration files.

### Supported Python Types

| Python Type             | JSON Equivalent         |
|-------------------------|-------------------------|
| <code>dict</code>       | Object <code>{}</code>  |
| <code>list</code>       | Array <code>[]</code>   |
| <code>str</code>        | String <code>""</code>  |
| <code>int</code>        | Number                  |
| <code>float</code>      | Number                  |
| <code>True/False</code> | <code>true/false</code> |
| <code>None</code>       | <code>null</code>       |

### Core API

| Function                  | Direction                   |
|---------------------------|-----------------------------|
| <code>json.dumps()</code> | Python object → JSON string |
| <code>json.loads()</code> | JSON string → Python object |
| <code>json.dump()</code>  | Python object → JSON file   |
| <code>json.load()</code>  | JSON file → Python object   |

### Limitations

- **No support** for `set`, `tuple` (silently becomes a list), `bytes`, `datetime`, or custom class instances.
- This is by design — JSON is meant to be **cross-language** (Java, C++, JavaScript, Go, etc. can all read it).

---

## 1.2 Pickle Serialization

**Pickle** is Python's native binary serialization protocol. Unlike JSON, it can serialize **almost any** Python object — including custom classes, tuples, sets, functions, and more.

### Core API

| Function                    | Direction                   |
|-----------------------------|-----------------------------|
| <code>pickle.dumps()</code> | Python object → bytes       |
| <code>pickle.loads()</code> | bytes → Python object       |
| <code>pickle.dump()</code>  | Python object → binary file |

| Function                   | Direction                   |
|----------------------------|-----------------------------|
| <code>pickle.load()</code> | binary file → Python object |

## Pickle vs JSON — Quick Comparison

| Feature        | JSON         | Pickle      |
|----------------|--------------|-------------|
| Format         | Text (UTF-8) | Binary      |
| Human-readable | Yes          | No          |
| Custom objects | No           | Yes         |
| Tuples / Sets  | No           | Yes         |
| Cross-language | Yes          | Python only |
| Security       | Safe         | ⚠️ Unsafe   |

⚠️ **Security Warning:** Never unpickle data received from an untrusted or unauthenticated source. A malicious pickle payload can execute arbitrary code during deserialization.

## Why Pickle Matters for Parallel Python

Python's `multiprocessing` module uses **pickle internally** to:

- Send function arguments to worker processes (via `Pool.map()`)
- Return results from workers back to the parent
- Transfer objects through `Queue` and `Pipe`

If an object **cannot be pickled**, it **cannot be sent** to another process. Understanding this saves hours of debugging "can't pickle" errors later.

## 🖥️ Demo Time

**Now, let's see these theoretical topics in action.**

- `01_json_serialization.py` — Serialize and deserialize Python data structures with JSON.
- `02_pickle_serialization.py` — Pickle complex objects including custom classes, and see the comparison table live.

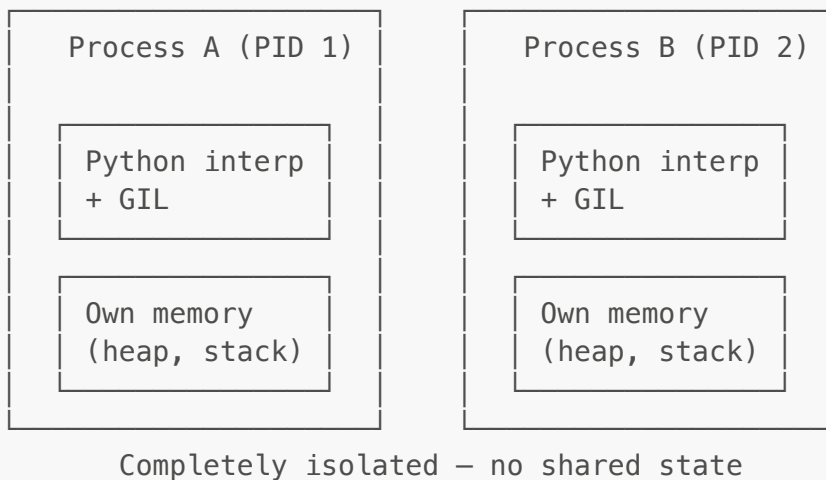
## 2. Multiprocessing — First Steps

What is a Process?

A **process** is an independent instance of a running program. Each process has:

- Its **own memory space** (heap, stack, global variables)
- Its **own Python interpreter** (including its own GIL — more on this later)

- A unique **Process ID (PID)** assigned by the operating system



## The `multiprocessing` Module

Python's `multiprocessing` module lets you spawn new processes, similar to `threading` but using **processes instead of threads**. Because each process has its own Python interpreter, processes can run truly in parallel on multi-core CPUs — they are not limited by the GIL.

### Creating a Process

```
import multiprocessing

def worker(name):
    print(f"Hello from {name}, PID = {os.getpid()}")

p = multiprocessing.Process(target=worker, args=("Worker-1",))
p.start()    # launch the process
p.join()    # wait for it to finish
```

Key methods:

| Method                  | What it does                               |
|-------------------------|--------------------------------------------|
| <code>start()</code>    | Fork/spawn a new process and begin running |
| <code>join()</code>     | Block until the process terminates         |
| <code>is_alive()</code> | Check if the process is still running      |

### Pool and Map — Data Parallelism Made Simple

When you have a **list of inputs** and want to apply the **same function** to each one, `multiprocessing.Pool` is the easiest approach:

```

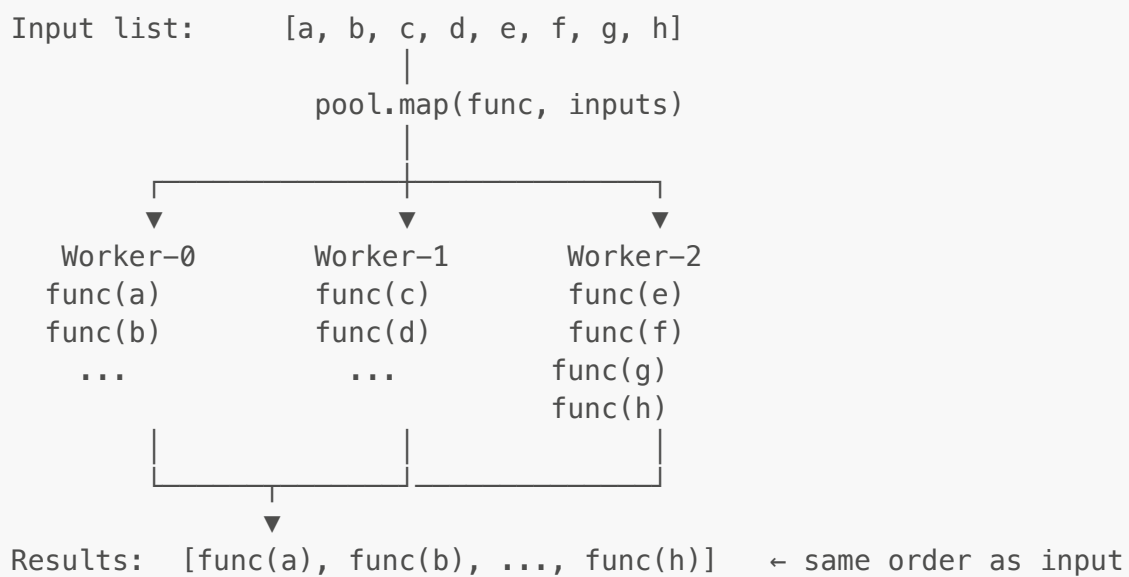
from multiprocessing import Pool

def square(x):
    return x * x

with Pool(processes=4) as pool:
    results = pool.map(square, [1, 2, 3, 4, 5])
# results = [1, 4, 9, 16, 25]

```

How `Pool.map()` works internally:



The pool automatically:

1. Creates N worker processes
2. Distributes chunks of the input to workers
3. Collects and **orders** the results
4. Returns them as a list in the **same order** as the input

## Demo Time

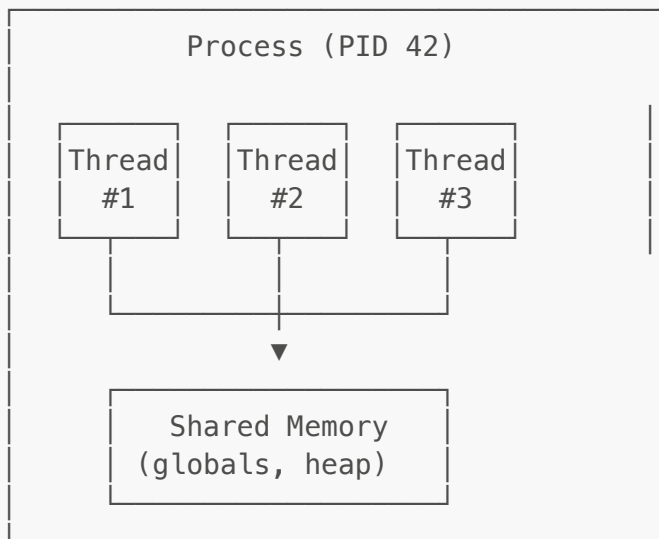
Now, let's see these theoretical topics in action.

- `03_simple_multiprocessing.py` — Spawn two processes computing squares and cubes concurrently; observe different PIDs and parallel execution time.
- `04_multiprocessing_pool_map.py` — Use `Pool.map()` to count primes in parallel; compare sequential vs parallel runtimes and observe real speedup.

## 3. Multithreading — First Steps

What is a Thread?

A **thread** is a lightweight unit of execution that exists **within** a process. Multiple threads share the **same memory space** — they can read and write the same variables, lists, and dictionaries directly.



## Threads vs Processes — Key Differences

| Property        | Thread                      | Process                    |
|-----------------|-----------------------------|----------------------------|
| Memory          | Shared (same address space) | Isolated (separate spaces) |
| Creation cost   | Low (lightweight)           | High (fork/spawn)          |
| Communication   | Direct variable access      | IPC (pipes, queues, etc.)  |
| GIL impact      | Contend for one GIL         | Each has its own GIL       |
| Crash isolation | One crash kills all threads | One crash ≠ other crashes  |

## The `threading` Module

```
import threading

def greet(name):
    print(f"Hello from {name}, TID = {threading.get_id()}")

t = threading.Thread(target=greet, args=("Worker",))
t.start()
t.join()
```

Notice that all threads share the **same PID** (they're in the same process) but each has a unique **thread identifier** (`threading.get_id()`).

## Communicating Between Threads

Since threads share memory, the simplest communication is a **shared variable**:

```
results = []    # shared list

def compute(n):
    results.append(n * n)    # all threads write to the same list

# Launch threads ...
# After join, `results` contains outputs from all threads
```

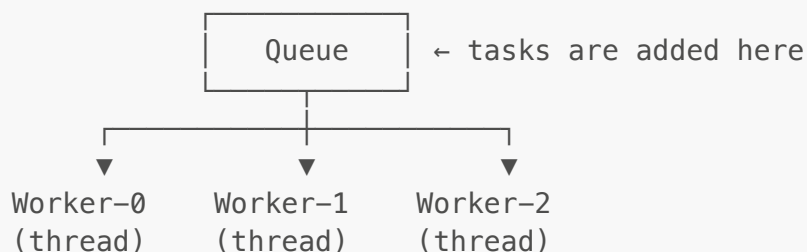
But for **producer-consumer** patterns and **bounded buffers**, use `queue.Queue` — it's **thread-safe** (internally locked), so you don't need to manage locks yourself:

```
import queue

q = queue.Queue()
q.put(item)    # producer adds work
item = q.get()    # consumer retrieves work (blocks if empty)
```

## The Worker Pool Pattern

A common pattern is to have a fixed number of worker threads pulling tasks from a shared queue:



Each worker runs an infinite loop:

1. Pull a task from the queue (blocks if empty)
2. Process the task
3. Repeat

To shut down workers gracefully, send a **sentinel value** (e.g., `None`) for each worker, or use `threading.Event` to signal cooperative shutdown.

## Graceful Shutdown with `threading.Event`

`threading.Event` provides a simple boolean flag that threads can check:

```

stop = threading.Event()

def worker():
    while not stop.is_set():      # run until signaled
        do_work()
        stop.wait(timeout=1.0)    # sleep, but wake up early if stop is set

# Later, from the main thread:
stop.set()    # signal the worker to stop

```

This is much better than a bare `while True + time.sleep()` loop because the thread responds **immediately** to the stop signal instead of sleeping through the full interval.

## 🖥️ Demo Time

Now, let's see these theoretical topics in action.

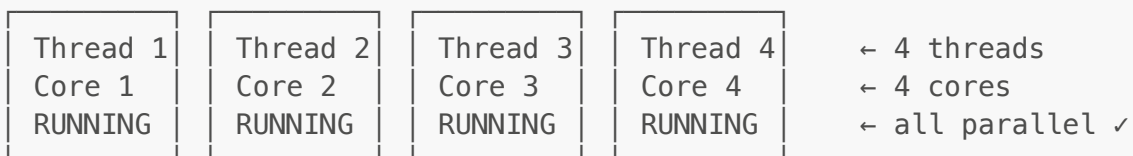
- `05_basic_multithreading.py` — Create threads with shared PIDs and unique TIDs; share a list across threads; build a worker pool pattern with a Queue.
- `06_thread_communication_stoppable.py` — Producer–consumer communication with `queue.Queue` and graceful shutdown of a long-running sensor thread using `threading.Event`.

## 4. Processes vs Threads and the Global Interpreter Lock

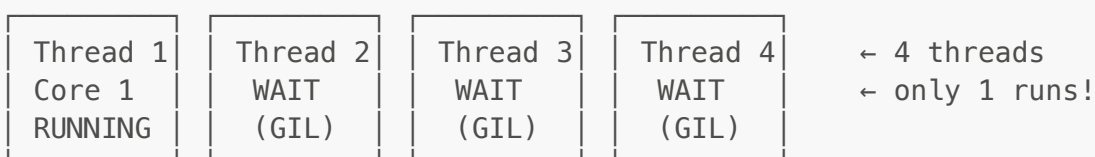
What is the GIL?

The **Global Interpreter Lock (GIL)** is a mutex (mutual exclusion lock) built into CPython (the standard Python implementation). It allows **only one thread** to execute Python bytecode at any given time, even on a multi-core machine.

Without GIL (what you'd expect):



With GIL (reality in CPython):



Why Does the GIL Exist?

1. **Memory management safety** — CPython uses reference counting for garbage collection. Without the GIL, every `Py_INCREF` / `Py_DECREF` would need its own lock, which would be slower overall.
2. **C extension compatibility** — Many C extensions assume they have exclusive access to Python objects. The GIL guarantees this.
3. **Simplicity** — A single global lock is simpler to implement and reason about than fine-grained locking.

## The GIL's Impact: CPU-Bound vs I/O-Bound

The GIL's impact depends entirely on the **type of work** your threads do:

### CPU-Bound Tasks (computation-heavy)

```
Thread 1:  [████ compute █████] [wait] [████ compute █████] [wait]...
Thread 2:  [wait] [████ compute █████] [wait] [████ compute █████]...
           ↑
       Only ONE thread computes at a time!
       Threads ≈ Sequential (no speedup)
```

- **Threads:** No speedup. The GIL serializes execution.
- **Processes:** Real speedup! Each process has its own GIL.

### I/O-Bound Tasks (network, disk, sleep)

```
Thread 1:  [request].....[response] [process]
Thread 2:      [request].....[response] [process]
Thread 3:          [request].....[response] [process]
           ↑
       GIL is RELEASED during I/O waits!
       Threads overlap I/O perfectly (speedup ✓)
```

- **Threads:** Good speedup! The GIL is **released** while waiting for I/O.
- **Processes:** Also work, but threads are cheaper (less memory, faster to create).

Summary: When to Use What

| Workload Type | Use Threads?    | Use Processes?     |
|---------------|-----------------|--------------------|
| CPU-bound     | ✗ No speedup    | ✓ Real parallelism |
| I/O-bound     | ✓ Great speedup | ✓ Works (heavier)  |
| Mixed         | Consider both   | Processes safer    |

## Sharing State: The Core Trade-off

### Threads: Shared Memory (Fast, but Risky)

Because threads share the same memory, one thread can directly read and modify variables that another thread is using. This leads to **race conditions**:

```
counter = 0

def increment():
    global counter
    for _ in range(100_000):
        counter += 1    # NOT atomic!
        # Internally: temp = counter → temp += 1 → counter = temp
        # Another thread can interleave between these steps!
```

If two threads run `increment()`, the final value of `counter` is **unpredictable** — it could be anything between 100,000 and 200,000.

### Solution: Use a Lock

```
lock = threading.Lock()

def safe_increment():
    global counter
    for _ in range(100_000):
        with lock:          # only one thread enters this block at a time
            counter += 1
```

### Processes: Isolated Memory (Safe, but Requires Effort)

Each process gets a **complete copy** of the program's memory. If a child process modifies a global variable, **the parent never sees the change**:

```
x = 0

def child():
    global x
    x = 42          # modifies the child's COPY
    print(x)        # prints 42

p = Process(target=child)
p.start(); p.join()
print(x)           # still 0 in the parent!
```

If you truly need shared state between processes, use:

- `multiprocessing.Value` — a single shared variable (int, float, etc.)
- `multiprocessing.Array` — a shared array
- `multiprocessing.Manager` — shared dicts, lists (slower, via a manager process)

All of these require **explicit locking** to avoid race conditions, just like threads.

### Demo Time

Now, let's see these theoretical topics in action.

- `07_gil_cpu_vs_io_benchmark.py` — Run the same workload with sequential, threaded, and multiprocessing execution for both CPU-bound and I/O-bound tasks. Observe the GIL's effect firsthand.
- `08_sharing_state_threads_vs_processes.py` — See a race condition happen live (threads without Lock), then fix it with Lock. Then see process memory isolation, and finally use `multiprocessing.Value` for explicit sharing.

## 5. Getting Started with Parallel Computing

### Why Parallel Computing?

The era of "just wait for faster CPUs" is over. Since around 2005, CPU clock speeds have plateaued at ~3–5 GHz due to physical limits (power dissipation, heat). Instead, hardware advances now come as **more cores**, not faster cores:

| Clock Speed Growth      | Core Count Growth   |
|-------------------------|---------------------|
| 1990: 40 MHz            | 1990: 1 core        |
| 2000: 1 GHz             | 2005: 2 cores       |
| 2005: 3.8 GHz           | 2010: 4–8 cores     |
| 2010: 3.5 GHz ← plateau | 2020: 16–64 cores   |
| 2024: 3.5 GHz ← same    | 2024: 64–128+ cores |

To exploit modern hardware, software **must** run in parallel.

### The Parallel Computing Memory Architecture

Computer architectures are classified by **Flynn's Taxonomy** based on how they handle **instruction streams** and **data streams**:

|                             | Data Streams |          |
|-----------------------------|--------------|----------|
|                             | Single       | Multiple |
| Instruction Streams: Single | SISD         | SIMD     |
| Multiple                    | MISD         | MIMD     |



### SISD — Single Instruction, Single Data

- Traditional **sequential** (von Neumann) computer
- One instruction operates on one piece of data at a time
- Example: a simple single-core CPU executing a program line by line

### SIMD — Single Instruction, Multiple Data

- **One instruction** is applied to **many data elements** simultaneously
- Example: GPU cores, Intel SSE/AVX vector instructions
- Ideal for: image processing, matrix operations, signal processing

SIMD Example: Add two arrays

A = [1, 2, 3, 4]

B = [5, 6, 7, 8]

Single ADD instruction

C = [6, 8, 10, 12] ← all four additions happen simultaneously

### MISD — Multiple Instruction, Single Data

- **Multiple instructions** operate on the **same data stream**
- Rare in practice; used in fault-tolerant systems (e.g., redundant flight computers processing the same sensor data through different algorithms)

### MIMD — Multiple Instruction, Multiple Data

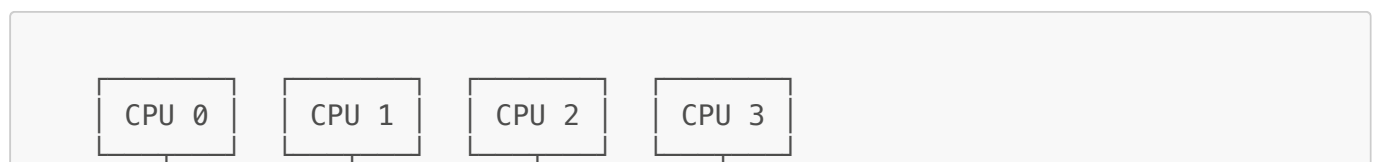
- **Multiple processors** execute **different instructions** on **different data** simultaneously
- This is the **most common** parallel architecture today
- Examples: multi-core CPUs, clusters of workstations, cloud computing

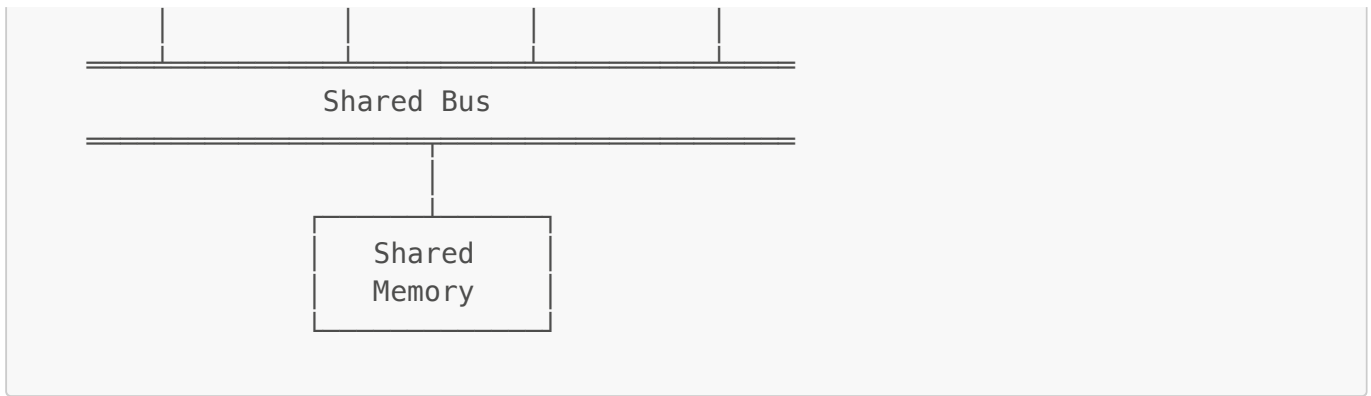
## Memory Organization

How processors **access memory** fundamentally shapes how parallel programs are designed:

### Shared Memory

All processors access a **common memory space** through a shared bus or interconnect:

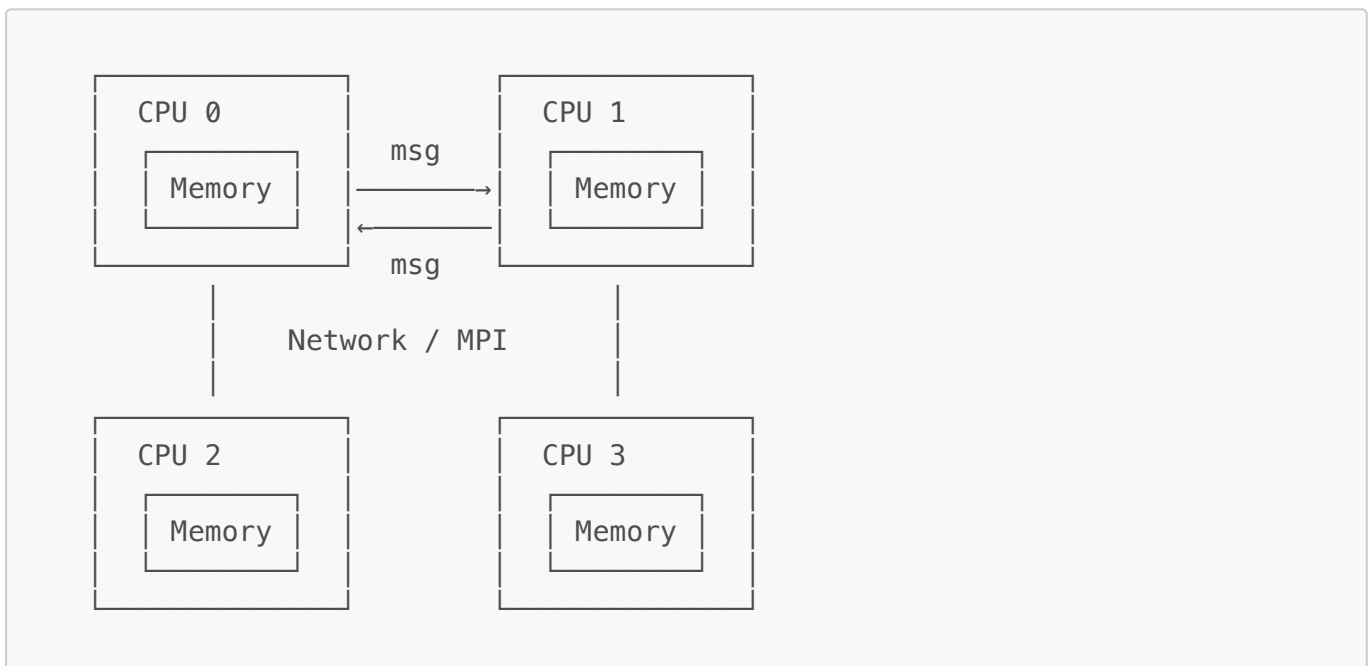




- **Pros:** Easy to program (just read/write shared variables)
- **Cons:** Bus contention, cache coherency overhead, doesn't scale beyond ~64–128 cores

### Distributed Memory

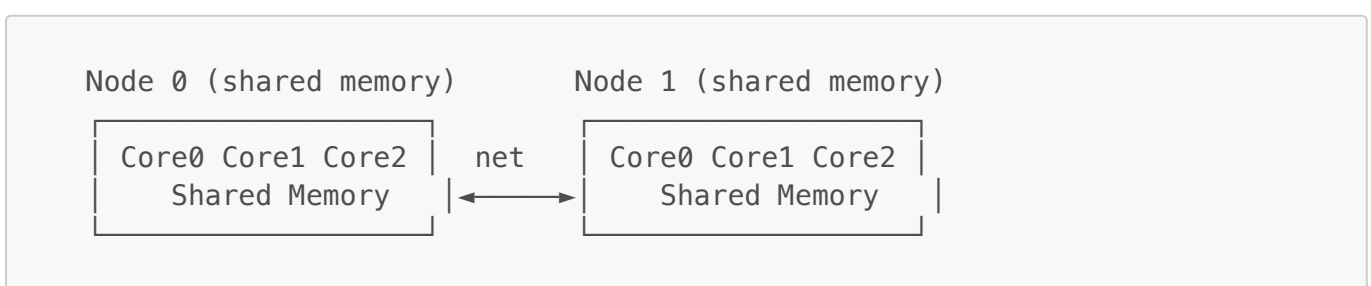
Each processor has its **own private memory**. Processors communicate by **sending messages** over a network:



- **Pros:** Scales to thousands of nodes, no cache coherency issues
- **Cons:** Programmer must explicitly manage all data movement (MPI)

### Hybrid: Clusters of Multi-Core Nodes

Modern HPC systems combine **both**: shared memory within each node, distributed memory across nodes.



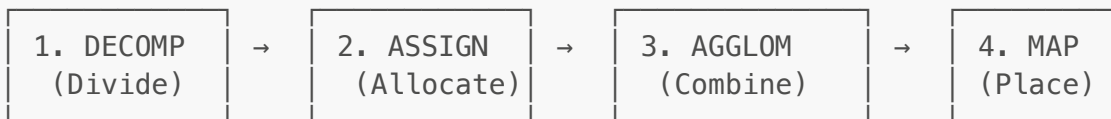
- Within a node → threads or multiprocessing (shared memory)
- Across nodes → MPI message passing (covered on Day 3)

## Parallel Programming Models

| Model           | Memory Model | Communication    | Python Module                 |
|-----------------|--------------|------------------|-------------------------------|
| Shared memory   | Shared       | Direct access    | <code>threading</code>        |
| Multithread     | Shared       | Shared variables | <code>threading</code>        |
| Message passing | Distributed  | Send/Receive     | <code>mpi4py</code> (Day 3)   |
| Data parallel   | Either       | Implicit         | <code>Pool.map()</code> , GPU |

## How to Design a Parallel Program

Designing a parallel program follows a four-step methodology called **Foster's Design Methodology**:



### Step 1: Task Decomposition

Break the problem into **small, independent tasks** that can run concurrently.

Problem: Process 1000 images

Decomposition:

```

Task 0: process image[0]
Task 1: process image[1]
...
Task 999: process image[999]
  
```

### Step 2: Task Assignment

Decide which tasks go to which processor/thread. The goal is **balanced load** — each processor should do roughly equal work.

### Step 3: Agglomeration

Combine small tasks into **larger chunks** to reduce overhead. Sending 1000 one-image tasks has more overhead than sending 10 hundred-image tasks.

Before agglomeration:  
[t0][t1][t2]...[t999]  
1000 tasks

After agglomeration:  
[t0-t99][t100-t199]...[t900-t999]  
10 chunks

#### Step 4: Mapping

Assign chunks to **physical processors**. Strategies include:

- **Static mapping:** Pre-assign chunks (simple, good for uniform tasks)
- **Dynamic mapping:** Workers pull tasks from a queue (good for non-uniform tasks)

Dynamic mapping variants:

| Strategy                           | Description                                          |
|------------------------------------|------------------------------------------------------|
| <b>Manager/Worker</b>              | One manager distributes tasks to workers on request  |
| <b>Hierarchical Manager/Worker</b> | Multiple levels of managers (for very large systems) |
| <b>Decentralized</b>               | Workers steal tasks from each other                  |

## 6. Performance Evaluation of Parallel Programs

How Do We Know If Our Parallel Program Is Good?

Three key metrics tell us how well we parallelized:

### 6.1 Speedup

**Speedup** measures how much faster the parallel version is compared to the sequential version:

$$S(N) = \frac{T_{\text{sequential}}}{T_{\text{parallel}}(N)}$$

where  $N$  is the number of processors.

| Speedup Value | Meaning                                          |
|---------------|--------------------------------------------------|
| $S(N) = 1$    | No improvement (same as sequential)              |
| $S(N) = N$    | <b>Linear speedup</b> (ideal / perfect)          |
| $S(N) > N$    | <b>Super-linear</b> (rare, due to cache effects) |
| $S(N) < 1$    | <b>Slowdown</b> (overhead > benefit)             |

### 6.2 Efficiency

**Efficiency** measures how well we're utilizing the processors:

$$E(N) = \frac{S(N)}{N} = \frac{T_{\text{sequential}}}{N \times T_{\text{parallel}}(N)}$$

| Efficiency        | Meaning                                  |
|-------------------|------------------------------------------|
| $E = 1.0$ (100%)  | Every processor is busy 100% of the time |
| $E = 0.5$ (50%)   | Half the processor time is wasted        |
| $E \rightarrow 0$ | Severe parallelization overhead          |

### 6.3 Amdahl's Law

**Amdahl's Law** gives the **theoretical maximum speedup** for a program with a serial (non-parallelizable) fraction  $S$ :

$$\text{Speedup}(N) = \frac{1}{S + \frac{1-S}{N}}$$

where:

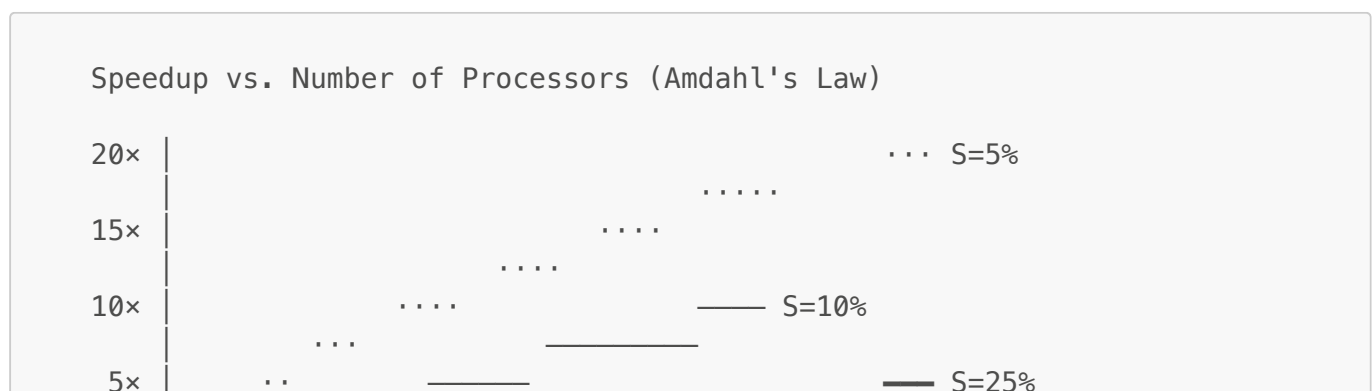
- $S$  = fraction of the program that is strictly serial ( $0 \leq S \leq 1$ )
- $N$  = number of processors
- $(1 - S)$  = parallelizable fraction

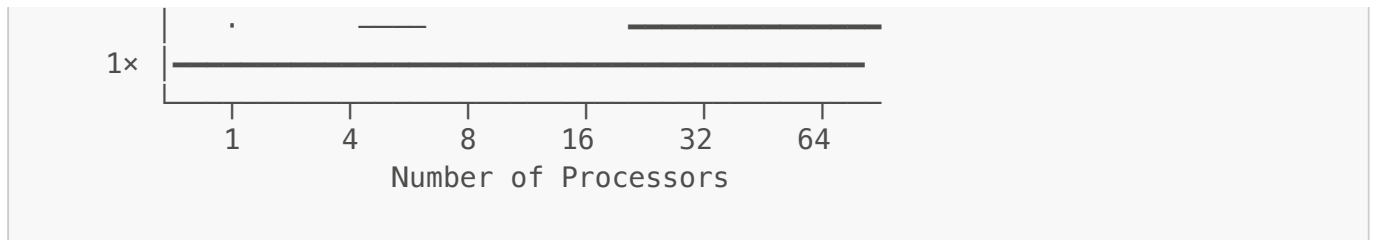
**Key insight:** Even with **infinite processors** ( $N \rightarrow \infty$ ):

$$\text{Max Speedup} = \frac{1}{S}$$

| Serial Fraction (S) | Max Speedup ( $N \rightarrow \infty$ ) |
|---------------------|----------------------------------------|
| 50%                 | 2×                                     |
| 25%                 | 4×                                     |
| 10%                 | 10×                                    |
| 5%                  | 20×                                    |
| 1%                  | 100×                                   |

A program with just **5% serial code** can never exceed **20×** speedup, no matter how many processors you throw at it. This is why optimizing the serial portions of code matters so much.





## 6.4 Gustafson's Law — A More Optimistic View

Amdahl's Law assumes a **fixed problem size**. But in practice, when we get more processors, we often **increase the problem size** to get better results (higher resolution, more data points, larger simulations).

**Gustafson's Law** measures **scaled speedup**:

$$S_{\text{scaled}}(N) = N - s \times (N - 1)$$

where  $s$  is the serial fraction of the **parallel** workload.

| Interpretation           | Amdahl                       | Gustafson                               |
|--------------------------|------------------------------|-----------------------------------------|
| Problem size             | Fixed                        | Scales with $N$                         |
| More processors means... | Diminishing returns          | Near-linear speedup                     |
| Outlook                  | Pessimistic                  | Optimistic                              |
| When to use              | "How fast can I solve THIS?" | "How BIG can I solve in the same time?" |

## Putting It All Together: Python in a Parallel World

Here's how Python's tools map to the concepts we've learned:

| Concept                     | Python Tool                                                                 |
|-----------------------------|-----------------------------------------------------------------------------|
| Process creation            | <code>multiprocessing.Process</code>                                        |
| Process pool                | <code>multiprocessing.Pool</code>                                           |
| Thread creation             | <code>threading.Thread</code>                                               |
| Thread synchronization      | <code>threading.Lock</code> , <code>Event</code> , <code>Semaphore</code>   |
| Inter-thread communication  | <code>queue.Queue</code>                                                    |
| Inter-process communication | <code>multiprocessing.Queue</code> , <code>Pipe</code> , <code>Value</code> |
| CPU-bound parallelism       | <code>multiprocessing</code> (bypass GIL)                                   |
| I/O-bound concurrency       | <code>threading</code> (GIL released during I/O)                            |
| Data parallelism            | <code>Pool.map()</code> , <code>Pool.starmap()</code>                       |
| Distributed parallelism     | <code>mpi4py</code> (Day 3), <code>Celery</code> (Day 4)                    |
| GPU parallelism             | <code>PyCUDA</code> , <code>PyOpenCL</code> (Day 5)                         |

## Demo Time

Now, let's see these theoretical topics in action.

- `09_performance_speedup_amdahl.py` — Measure sequential vs parallel runtimes across 1, 2, 4, ... N cores. Compute actual Speedup and Efficiency, and compare against theoretical Amdahl's Law and Gustafson's Law predictions.

## Day 1 Summary

| What We Learned             | Key Takeaway                                                                                                                                 |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Serialization</b>        | JSON (cross-language, text) vs Pickle (Python-only, binary, supports all types). Pickle is used internally by <code>multiprocessing</code> . |
| <b>Processes</b>            | Independent memory, own GIL → true parallelism for CPU-bound work                                                                            |
| <b>Threads</b>              | Shared memory, one GIL → great for I/O-bound work, limited for CPU-bound                                                                     |
| <b>GIL</b>                  | Only one thread runs Python bytecode at a time. Use processes to bypass it.                                                                  |
| <b>Race conditions</b>      | Shared data + concurrent writes = bugs. Use Locks.                                                                                           |
| <b>Flynn's Taxonomy</b>     | SISD, SIMD, MISD, MIMD — MIMD is the most common today                                                                                       |
| <b>Memory models</b>        | Shared memory (threads, multiprocessing) vs Distributed memory (MPI)                                                                         |
| <b>Foster's methodology</b> | Decompose → Assign → Agglomerate → Map                                                                                                       |
| <b>Performance metrics</b>  | Speedup, Efficiency, Amdahl's Law (pessimistic), Gustafson's Law (optimistic)                                                                |

## What's Coming Tomorrow (Day 2)

Day 2 dives deeper into **shared-memory parallelism**:

- Advanced thread synchronization: Lock, RLock, Semaphore, Condition, Event
- Thread communication with queues
- Full process lifecycle: spawn, name, background, kill
- Inter-process communication: Queues, Pipes
- Shared state with `Value` and `Array`
- Process pools

## End of Day 1